

NUS Orbital 2026



Milestone 01

[Poster](#) [Video](#) [App](#) [Logs](#)

Atlas (Team ID:7660)

Wang Qichen (A0337219B)

Feng Yi (A0322277E)

Proposed Level of Achievement:

Artemis

Contents

Deployment	4
Motivation	4
Our Aim	4
User Stories	4
Overall Architecture Diagram	6
Core Features	7
Feature Ideation	9
Repository and Project Foundation	11
Project Structure and Version Control	11
Project-Level Software Engineering	12
GitHub Project Management	12
Engineering Principles	13
Risk Management	13
Authentication and Protected Access	14
Email, Demo, and Google Sign-In	14
Protected Routes	15
Authentication Sequence Diagram	15
Campus Map Rendering	16
MapLibre and OSM Tiles	16
Route Tracing	16
Daily Assistant, Facilities, and Schedule	18
Facility Discovery	18
Schedule Management	18
Recommendation Cards	19
NUSMods Sync and Bus Context	20
NUSMods Sync	20
Bus and Transit Context	20

Mobile Frontend Experience	22
Responsive Layout	22
Standardised Software Diagrams	23
ER Diagram	23
Activity Diagram	24
Automated Testing and QA	25
Automated Checks	25
Manual Test Matrix	26
CI/CD Pipeline	27
Continuous Integration Gates	27
Continuous Deployment Path	28
0.1 Unit Testing	28
0.2 Software Design Principles	29
Milestone 01 Progress	30
Planned Next Steps	30

Deployment

The deployed web application is available at: orbital-artemis-armap-nus.onrender.com

Motivation

Navigating through the NUS campus can be difficult because complete geospatial and indoor facility data is not always publicly available or intuitive to use. Even after reaching the correct building, students may still need to locate lecture rooms, tutorial classrooms, study spaces, restrooms, lifts, printing services, and other facilities.

These navigation needs are closely tied to students' daily routines and schedules. Atlas is motivated by the need for a smarter, more context-aware campus assistant that supports both outdoor movement across NUS and indoor discovery within selected buildings.

Our Aim

Atlas aims to create a seamless campus assistant for NUS students. The project combines map-based navigation, facility discovery, schedule-aware guidance, and a daily assistant interface so that students can move through campus more confidently.

For Milestone 01, our goal is to establish a working full-stack prototype: authentication, a protected dashboard, campus data APIs, map rendering, location search, and deployment. Later milestones will extend this foundation toward richer AR-style navigation, indoor guidance, and smarter route planning.

User Stories

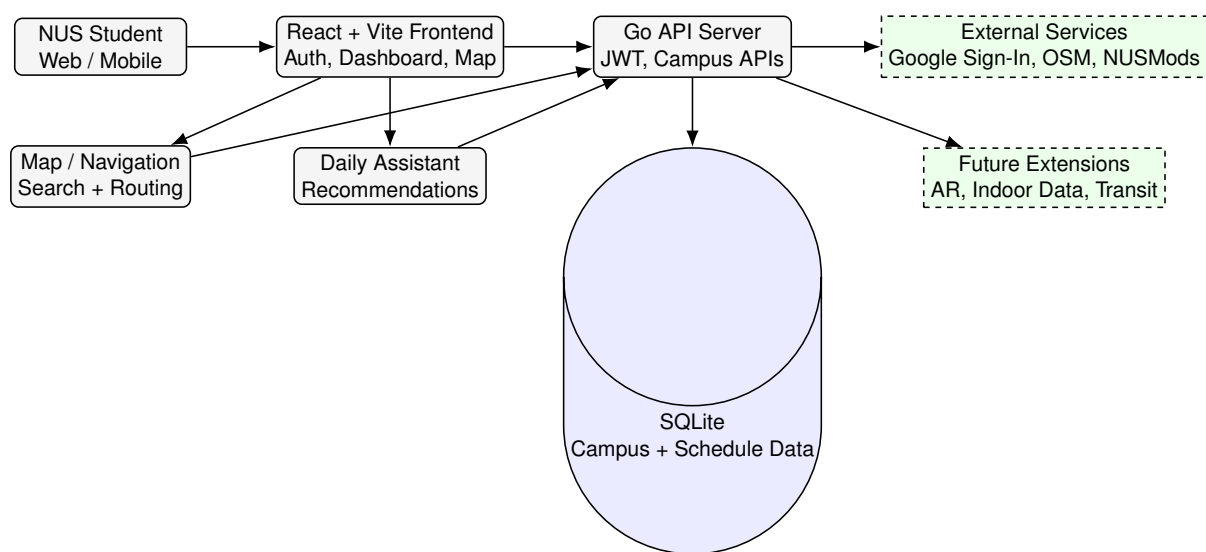
1. As an NUS student, I can use the AR map to navigate from my current location to the correct building and classroom so that I can arrive at class on time.
2. As an NUS student unfamiliar with a campus building, I can locate indoor facilities such as restrooms, lifts, study spaces, and printing services so that I can navigate the building more easily.
3. As an NUS student, I can receive location-aware reminders and suggestions based on my schedule and current position so that I can manage my day more smoothly.
4. As a freshman unfamiliar with NUS, I can search for lecture theatres, tutorial rooms, and laboratories so that I can navigate the campus confidently.
5. As an exchange student visiting NUS for the first time, I can follow AR navigation instructions instead of static maps so that navigation feels more intuitive.
6. As a student rushing between consecutive classes, I can estimate travel time between buildings so that I know when to leave for class.
7. As a student studying late on campus, I can locate nearby study spaces and facilities through AR guidance so that I can find suitable places quickly.

8. As a wheelchair user, I can find wheelchair-friendly routes, accessible entrances, and lifts so that I can travel across campus more comfortably.
9. As a student unfamiliar with NUS bus routes, I can locate nearby shuttle bus stops and view available shuttle services so that I can travel more efficiently.
10. As a student attending events on campus, I can receive recommendations for nearby ongoing events and activities so that I can participate more conveniently.
11. As a student looking for a quiet place to study, I can receive recommendations for less crowded study areas and facilities so that I can study more effectively.
12. As a student entering a large building for the first time, I can receive indoor AR guidance to classrooms, offices, and facilities so that I do not need to rely on physical signboards.
13. As a student who frequently prints documents on campus, I can quickly locate nearby printing services so that I can save time.
14. As a student concerned about safety, I can locate emergency exits, first-aid points, and security offices within campus buildings so that I can respond quickly during emergencies.
15. As a student contributing to the campus community, I can add or update map information such as facilities, pathways, and corrections so that the map remains accurate and useful.
16. As a student with back-to-back classes, I can receive reminders on when to leave my current location so that I can arrive on time for my next lesson.
17. As a visitor attending an event at NUS, I can navigate to specific venues and facilities so that I can find locations without prior knowledge of the campus layout.
18. As a student using the system daily, I can save frequently visited locations so that I can access navigation more quickly.
19. As a student who wants to avoid crowded routes, I can view alternative paths and facility options so that I can choose a route that matches my comfort and time constraints.
20. As a campus administrator maintaining navigation data, I can review user-submitted map corrections before they are published so that the system remains accurate and trustworthy.

Overall Architecture Diagram

Atlas uses a **client-server layered architecture**. The client is a React/Vite frontend responsible for user interaction, map display, authentication screens, dashboard workflows, and mobile-responsive layouts. The server is a Go API responsible for authentication, JWT issuing and validation, campus data access, schedule APIs, external API integration, and database persistence. Within the client and server, responsibilities are further separated into layers such as UI components, map services, API handlers, storage, and external-service clients.

This architecture is used because Atlas is expected to grow beyond a small prototype. A campus assistant needs to combine user-facing workflows, protected user data, campus data, maps, and external services. Keeping these concerns in separate layers makes the system easier to test, easier to deploy, and safer to extend in later milestones. For example, future AR or indoor-navigation modules can be added to the map/navigation layer without rewriting authentication or database code.



Atlas overall architecture for Milestone 01 and planned Artemis extensions

Core Features

The feature list below is based on the GitHub issue tracker, current repository implementation, and the product direction described in the original proposal. For Artemis Milestone 01, Atlas is treated as more than a small demo: it is a full-stack foundation for campus navigation, schedule-aware assistance, and future AR/indoor extensions.

S/N	Feature	Completed At	Brief Explanation
Core Features			
1	Authentication and protected access	Milestone 01	Users can enter Atlas through email/password login, demo login, or Google Sign-In, then access protected dashboard and map workflows.
2	Interactive campus map	Milestone 01	Users can view an NUS-centered MapLibre map with campus markers and route layers.
3	Location search	Milestone 01	Users can search for places, select a result, and center the map on the chosen destination.
4	Route tracing	Milestone 01	Users can preview a route line between selected start and end points on the web map.
5	Daily assistant recommendations	Milestone 01	Users can view recommendation cards that connect schedule context, campus data, and likely next actions.
6	Facility discovery and filtering	Milestone 01	Users can filter seeded buildings and facilities by building code and facility type, such as study spaces, restrooms, and printers.
7	Schedule management	Milestone 01	Users can view, create, and delete schedule items so the assistant can reason about upcoming locations.
8	Campus data sync status	Milestone 01	Users can see whether external campus/timetable data has been synced and trigger a manual sync during demonstration.
9	Shuttle bus arrival dashboard	Milestone 01 / Extension	Users can view shuttle stops, route pickup points, arrival times, and active bus context through backend-proxied bus APIs.
10	Mobile-responsive usage	Milestone 01	Users can access the same authentication, dashboard, and map workflows on smaller mobile viewports without horizontal scrolling.

Repository setup, monorepo structure, GitHub issues, and the iOS shell are treated as **engineering support**, not user-facing features. They are documented later under project-level

software engineering because they explain how the team builds and maintains the app.

Feature Ideation

The ideation below records the user problem, proposed solution, and engineering direction for all 10 user-facing features. The engineering direction column also explains **why** the direction fits Atlas, since the aim is not only to list technologies but to justify design choices.

S/N	Feature	User Problem / Ideation	M1 Engineering Direction and Rationale
1	Authentication and protected access	A campus assistant may eventually contain schedule and preference data, so users need a trusted entry point.	Implement email/password login, demo mode, Google Sign-In verification, JWT issuing, and protected routes. JWT is suitable because the backend can issue a signed token after verification and later validate protected requests without storing a server-side session for every page visit.
2	Interactive campus map	Students need an interactive spatial view of campus rather than static text instructions.	Use MapLibre with OpenStreetMap tiles because it gives an open-source map renderer that can accept custom campus layers. This matters because later AR/indoor work will need Atlas-owned overlays rather than only a fixed third-party map screen.
3	Location search	Users should not need to pan manually across campus to find lecture theatres, tutorial rooms, or buildings.	Separate geocoding/search services from the UI. This keeps search logic reusable for the map page, dashboard recommendations, and future saved-location features.
4	Route tracing	Students rushing between classes need to preview how movement between places may look.	Draw route lines through a map-engine abstraction. This is important because route display, bus routes, and future AR overlays all need to modify map layers without duplicating MapLibre source/layer setup code.
5	Daily assistant recommendations	Navigation decisions depend on schedule, next destination, facility availability, and sync state.	Combine schedule cards, recommendation cards, building context, facilities, and sync status in one dashboard. This is the best M1 direction because the team can test the daily workflow before adding more complex ranking or personalization.
6	Facility discovery and filtering	Indoor needs such as toilets, lifts, printers, and study areas are often discovered only after entering a building.	Store buildings and facilities as separate SQLite entities and expose protected API filters. This avoids duplicating building data for every facility and prepares the schema for richer indoor/floor data later.

7	Schedule management	Schedule-aware navigation requires structured upcoming events rather than hard-coded demo text.	Provide API and UI support for reading, adding, and deleting schedule items. Storing schedule items on the backend makes them available to recommendations, route preparation, and future NUSMods sync.
8	Campus data sync status	A real NUS assistant should eventually update from official academic data instead of relying only on static seed data.	Add sync status endpoints, a manual sync trigger, and scheduler configuration. This makes external-data health visible to users and gives the team a controlled way to test sync behavior before full automation.
9	Shuttle bus arrival dashboard	Campus movement is multimodal: students choose between walking and shuttle buses depending on time and location.	Keep bus API calls on the backend and normalize stops, routes, pickup points, and arrival times before sending data to the frontend. Server-side proxying is preferred because it avoids exposing bus credentials and allows caching/fallback behavior.
10	Mobile-responsive usage	Campus navigation is primarily mobile, so desktop-only usability is insufficient.	Use responsive layouts and manual viewport testing for auth, dashboard, map, and mobile screenshots. This is important because the core use case happens while students are moving around campus, often on phones.

Repository and Project Foundation

This section is not counted as a user-facing feature. It documents the engineering foundation that allows the team to build the user-facing features in a maintainable way: reproducible setup instructions, a clear project structure, issue tracking, and deployment configuration.

Project Structure and Version Control

Atlas is organized into a frontend, backend, deployment configuration, and supporting documentation. This mirrors the modular design described in the proposal and keeps responsibility boundaries clear. The frontend owns user interaction, the backend owns authentication and campus data APIs, and the deployment files define how the system runs in production.

Atlas uses a monorepo because the frontend, backend, deployment files, poster, and milestone report are tightly connected during M1. For a small two-person team, a monorepo reduces coordination overhead: one issue or pull request can show the full change across UI, API, documentation, and deployment. The trade-off is that the repository contains several types of work in one place, but this is acceptable because the project is still small and the folder boundaries are explicit.

```
Orbital_Artemis_ArMap_Nus/
|-- frontend/ React + Vite frontend and mobile app shell
| |-- src/ Auth, dashboard, campus map, and UI code
| '-- ios/ Capacitor iOS project
|-- backend/ Go API server
| |-- cmd/server/ Server entry point
| '-- internal/atlas/ Auth, campus data, routing, and API logic
|-- deploy/ Production deployment configuration
|-- poster_work/ Poster and presentation assets
|-- Dockerfile Container build definition
|-- docker-compose.yml Local multi-service run configuration
'-- readme.tex Milestone report source
```

Project Structure

The repository uses GitHub issues and pull requests as progress logs. Issue #1 initialized the repository and documentation baseline, while later issues track authentication, map rendering, search, dashboard integration, and mobile usability. This supports the proposal's branch-based workflow: tasks are decomposed into issues, implemented on feature branches, reviewed through pull requests, and eventually integrated into `main`.

The main trade-off of this structure is that it creates more files and coordination overhead than a single-folder prototype. However, the benefit is significant for an Artemis-level project: each module can evolve independently, deployment can be reproduced, and later AR/navigation extensions can be added without rewriting the whole codebase.

Project-Level Software Engineering

Atlas applies software engineering practices at the project level, not only inside individual files. The goal is to make the project trackable, reviewable, testable, and deployable throughout Orbital.

GitHub Project Management

GitHub issues are used as the team progress log. Each issue represents a concrete feature, defect, or engineering task, such as authentication, protected routing, map rendering, mobile layout, dashboard integration, and map polygon fixes. The public issue tracker functions as both a task board and an audit trail for mentor review.

The intended workflow is:

1. Break milestone requirements into GitHub issues with clear acceptance criteria.
2. Assign work to a team member and connect it to the relevant feature area.
3. Implement changes on a focused branch instead of mixing unrelated work.
4. Review the change through a pull request before merging to `main`.
5. Use the issue and pull request history as progress logs for milestone submission.

This workflow is intentionally heavier than a quick prototype workflow. It is justified because Artemis projects are expected to demonstrate consistent engineering discipline and maintainability, not only visible screens.

GitHub Projects Tab Screenshot Placeholder

Add screenshot at `readme_assets/github-projects-tab.png`

GitHub Projects tab used to track milestone tasks, statuses, and review progress

Engineering Principles

Principle	How Atlas Applies It
Separation of concerns	Frontend pages and UI components are separated from backend API handlers, authentication logic, storage logic, and deployment configuration.
API-first integration	Frontend features call documented backend endpoints such as /api/buildings, /api/facilities, /api/schedule, /api/recommendations, and bus endpoints.
Progressive delivery	The team implements a reliable 2D map and schedule-aware dashboard before adding higher-risk AR and indoor navigation layers.
Security by default	Protected pages require authentication, backend data routes are guarded with JWT middleware, and Google ID token verification happens server-side.
Reproducibility	Local setup, environment variables, Docker build, Docker Compose deployment, and production Caddy configuration are documented in the repository.
Graceful degradation	Demo credentials, seeded data, and bus demo fallback data allow reviewers to evaluate the project even without private external API keys.

Risk Management

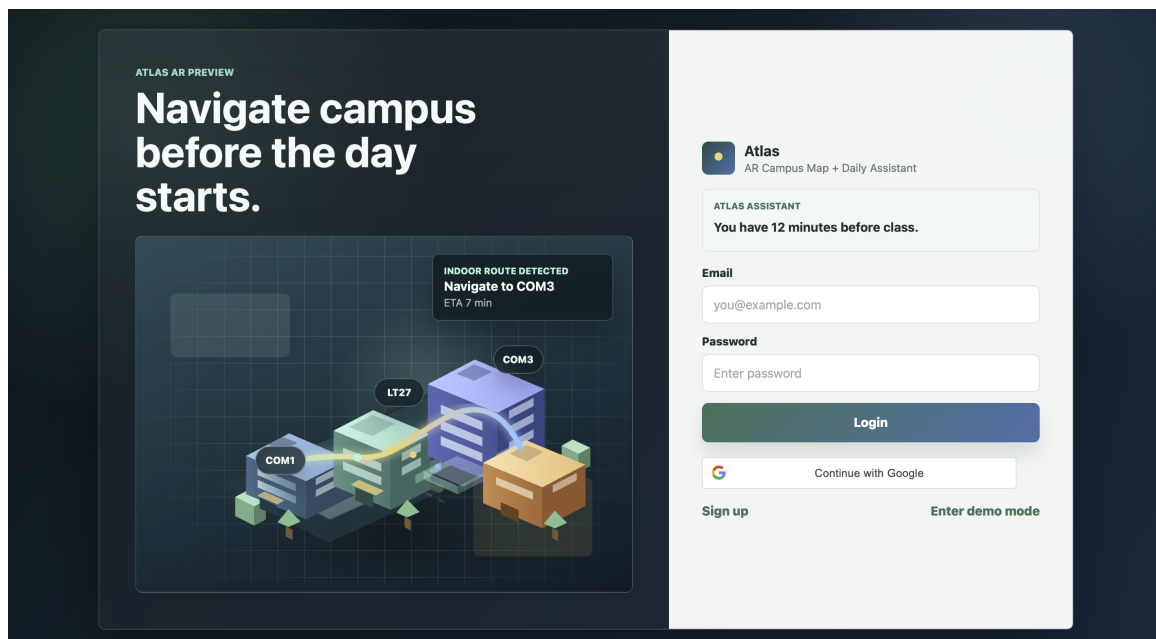
Risk	Impact	Mitigation
Incomplete campus/indoor data	The assistant may not cover every building or facility at M1.	Use seeded demo data for core flows, design APIs around building/facility models, and expand coverage in later milestones.
External API reliability	NUSMods, Google verification, map tiles, or bus APIs may fail or require credentials.	Keep configuration in environment variables and provide demo/fallback behavior where possible.
AR complexity	Full AR navigation may exceed M1 implementation capacity.	Build a stable map, routing, and data foundation first, then integrate AR overlays after core navigation behavior is reliable.
Mobile usability defects	Navigation apps are frequently used on phones, where cramped layouts can break flows.	Track responsive layout as a feature, include mobile screenshots, and test common auth/dashboard/map flows on small viewports.

Authentication and Protected Access

Authentication allows students to enter Atlas through email/password login, registration, demo mode, or Google Sign-In. Once authenticated, users can access protected pages such as the dashboard and map.

Email, Demo, and Google Sign-In

The login system supports multiple entry paths. Email/password login provides a traditional account flow, demo mode allows quick testing without setup, and Google Sign-In supports a more realistic student login experience.



Authentication Screen

On the backend, Atlas verifies credentials and issues a JWT. For Google Sign-In, the frontend obtains a Google ID token and sends it to the backend, where it is verified against the configured Google client ID. If verification succeeds, the backend returns the same application JWT format used by email login. This keeps the rest of the application independent of the authentication method.

JWT is used because it is a signed token format with a header, payload, and signature. In Atlas, the payload can carry basic claims such as the authenticated user's identity and expiry time, while the signature allows the backend to detect tampering. This fits the project because protected API routes can validate a request from the `Authorization` header without requiring the frontend to resend the password or requiring every API handler to implement its own login logic.

From a software engineering perspective, the design demonstrates separation of concerns and a form of security boundary. The frontend handles user interaction and stores the returned application token, while the backend performs trust-sensitive verification, password checking, Google ID token verification, and JWT signing. The main trade-off is external dependency: Google Sign-In requires OAuth configuration and network access to Google's verification service during development and deployment. To reduce this friction for M1 evaluation, Atlas also

keeps email/password and demo login paths available.

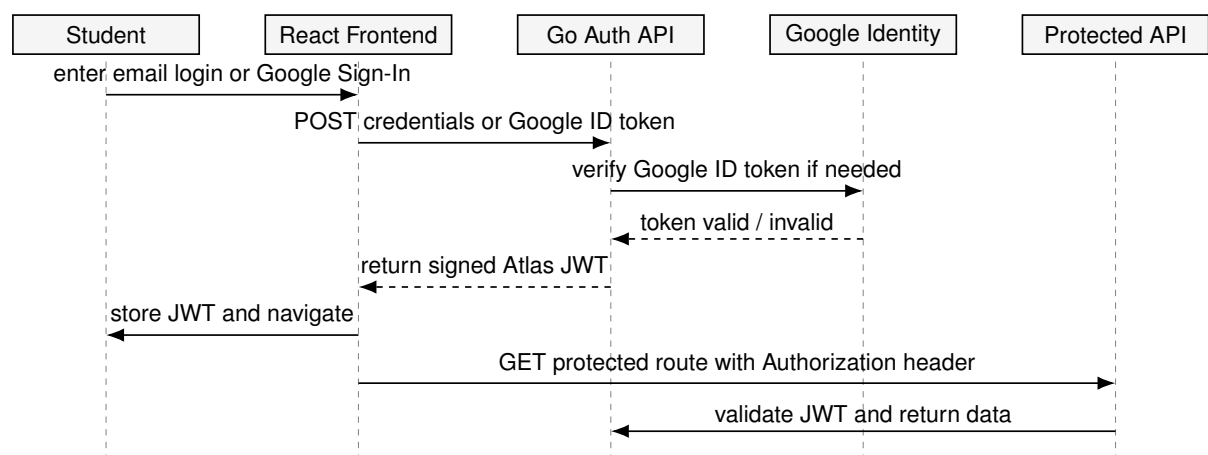
Protected Routes

Protected routes guard the dashboard and map pages. If a user is not authenticated, the frontend redirects them to the login screen. API requests also attach the JWT in the `Authorization` header, allowing the backend to reject unauthenticated access consistently.

This pattern reduces duplicated access checks. Instead of manually checking login state inside every page component, protected routing creates a reusable access boundary. It also makes later feature additions easier, because new protected pages can reuse the same wrapper.

Authentication Sequence Diagram

The sequence diagram below is placed in the authentication section because it explains the exact interaction order for email login, Google Sign-In, JWT issuing, and protected route access. The key design point is that the frontend may collect credentials or a Google ID token, but the backend remains the authority that verifies identity and issues the Atlas application JWT.



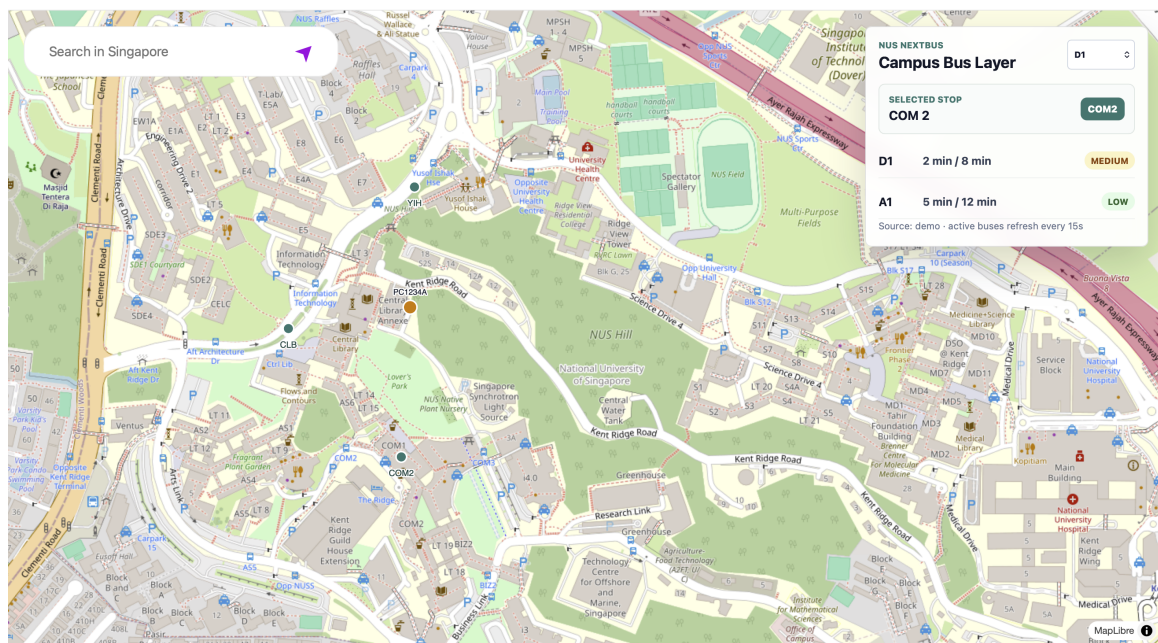
Sequence diagram for login, Google Sign-In, JWT issuing, and protected route access

Campus Map Rendering

The map feature gives users an interactive visual interface for campus navigation. For Milestone 01, Atlas renders a MapLibre/OpenStreetMap-based campus map centered around NUS and prepares the map surface for future AR and indoor navigation work.

MapLibre and OSM Tiles

The frontend uses MapLibre to render map tiles and manage markers, routes, and future campus overlays. OpenStreetMap (OSM) tiles are small map pieces that together form the visible base map, similar to how a large map is loaded as many smaller grid sections. MapLibre is used because it is an open-source renderer that lets Atlas add custom layers such as campus markers, bus routes, and future indoor/AR overlays.



Campus Map Screen

The map logic is separated into a map initialization module, a MapEngine helper, routing services, and UI components. This modular approach prevents the main map page from becoming a single large component. It also makes it easier to add bus layers, indoor facility overlays, or AR-style markers later.

The bug tracked in issue #17 showed why this separation matters. The issue involved map building outlines and layer rendering. Such map bugs can come from coordinate format differences, layer ordering, or adding a layer before its data source is ready. By separating map engine actions from UI state, Atlas can debug and update rendering logic without changing authentication or dashboard code.

Route Tracing

Route tracing allows selected start and end points to be drawn on the map. In Milestone 01, this is implemented as a 2D route line on the MapLibre layer. This is an implementation stage toward the proposal's broader AR navigation goal rather than a complete AR feature.

The engineering reason for implementing 2D tracing first is risk control. Before adding camera-based AR guidance, the team needs reliable location search, route data, marker placement, and map-layer management. If these foundations are unstable, an AR overlay would only make debugging harder.

Daily Assistant, Facilities, and Schedule

The daily assistant dashboard connects campus navigation to student routines. Instead of treating the map as a standalone feature, Atlas combines schedule items, next-location context, building data, facility filters, and recommendation cards into one student-facing workflow.

Facility Discovery

Facility discovery focuses on practical indoor and near-indoor needs: study areas, lifts, restrooms, and printing locations. In Milestone 01, the backend seeds representative NUS building and facility data in SQLite and exposes it through protected API endpoints.

Endpoint / Data	Purpose
GET /api/buildings	Returns campus buildings with code, name, description, coordinates, floor count, and indoor support status.
GET /api/facilities	Returns facilities and supports filters such as <code>building=COM1</code> and <code>type=study_space</code> .
Seeded demo data	Allows evaluators to test building and facility discovery without requiring private production data.

The main design decision is to model facilities separately from buildings. This allows Atlas to add many facility records under one building and later attach richer indoor information such as floor plans, accessibility paths, or AR markers.

This design follows a simple data-normalisation principle: building information such as code, name, coordinates, and floor count should not be repeated inside every facility record. Facilities keep only facility-specific fields such as name, type, floor, crowd level, and the building reference. This makes filtering easier and reduces the risk that one building is described inconsistently across multiple facility rows.

Schedule Management

Schedule management is included because a campus assistant becomes more useful when it understands where the student needs to go next. The current prototype supports viewing, creating, and deleting schedule items. Each item can include a location code, allowing recommendation cards to connect time-based context with campus data.

This feature also prepares the app for future NUSMods integration. Rather than hard-coding a static timetable into the frontend, the backend exposes schedule APIs and stores schedule data in SQLite. This gives the team a stable place to add timetable sync, conflict warnings, and route preparation in later milestones.

The engineering boundary is intentionally API-first. The frontend does not directly manipulate the database; it sends create/delete/read requests to backend schedule endpoints. This keeps validation and persistence in one place and allows future recommendation logic to reuse the same schedule data without duplicating state in separate frontend components.

Recommendation Cards

Recommendation cards summarize useful actions such as where the next class is, which building context matters now, or which facilities are likely relevant. In M1, the recommendations are intentionally simple and explainable. This helps the team test the user workflow before adding more advanced ranking or personalization.

The current recommendation design favours deterministic rules over a black-box ranking model. This is appropriate for M1 because the team needs to verify whether the dashboard workflow is useful before optimising recommendation quality. It also makes testing easier: if the schedule or building context changes, the expected recommendation can be reasoned about directly.

NUSMods Sync and Bus Context

Atlas includes early integration points for external campus systems. These are important Artemis extensions because they move the project beyond a static campus map and toward a live assistant.

NUSMods Sync

The backend contains configuration for academic year, sync interval, HTTP timeout, and NUSMods integration. The dashboard exposes sync status and a manual sync trigger so users can see whether external data integration is healthy.

Sync status is modelled separately from schedule items and facilities because it describes the health of external data integration, not the data itself. This separation lets the dashboard tell users whether the app is using fresh, stale, failed, or manually seeded data without mixing operational metadata into user schedule records.

Endpoint / Setting	Purpose
GET /api/sync/status	Shows the latest sync state for campus or timetable data.
POST /api/sync/run	Allows a manual sync trigger during development and demonstration.
NUSMODS_ACAD_YEAR	Selects the academic year used by the NUSMods client.
SYNC_INTERVAL	Controls the scheduler interval for future automatic syncing.

Bus and Transit Context

Campus movement often involves a choice between walking, waiting for a shuttle, or combining both. Atlas therefore includes bus context as an Artemis extension feature. The backend supports endpoints for bus stops, routes, arrivals, active vehicles, alerts, and context near a latitude/longitude point.

The current design keeps bus data server-side. This avoids exposing private API keys in the browser and allows the backend to cache, normalize, or fall back to demo data. For M1, this establishes the integration boundary even if later milestones refine the frontend presentation and multimodal routing logic.

The bus module uses the same engineering principle as other external integrations: the frontend consumes Atlas backend endpoints, while the backend handles third-party API credentials, response normalisation, caching, and fallback behavior. This makes the user-facing dashboard more stable because it does not need to understand every raw bus API response shape.

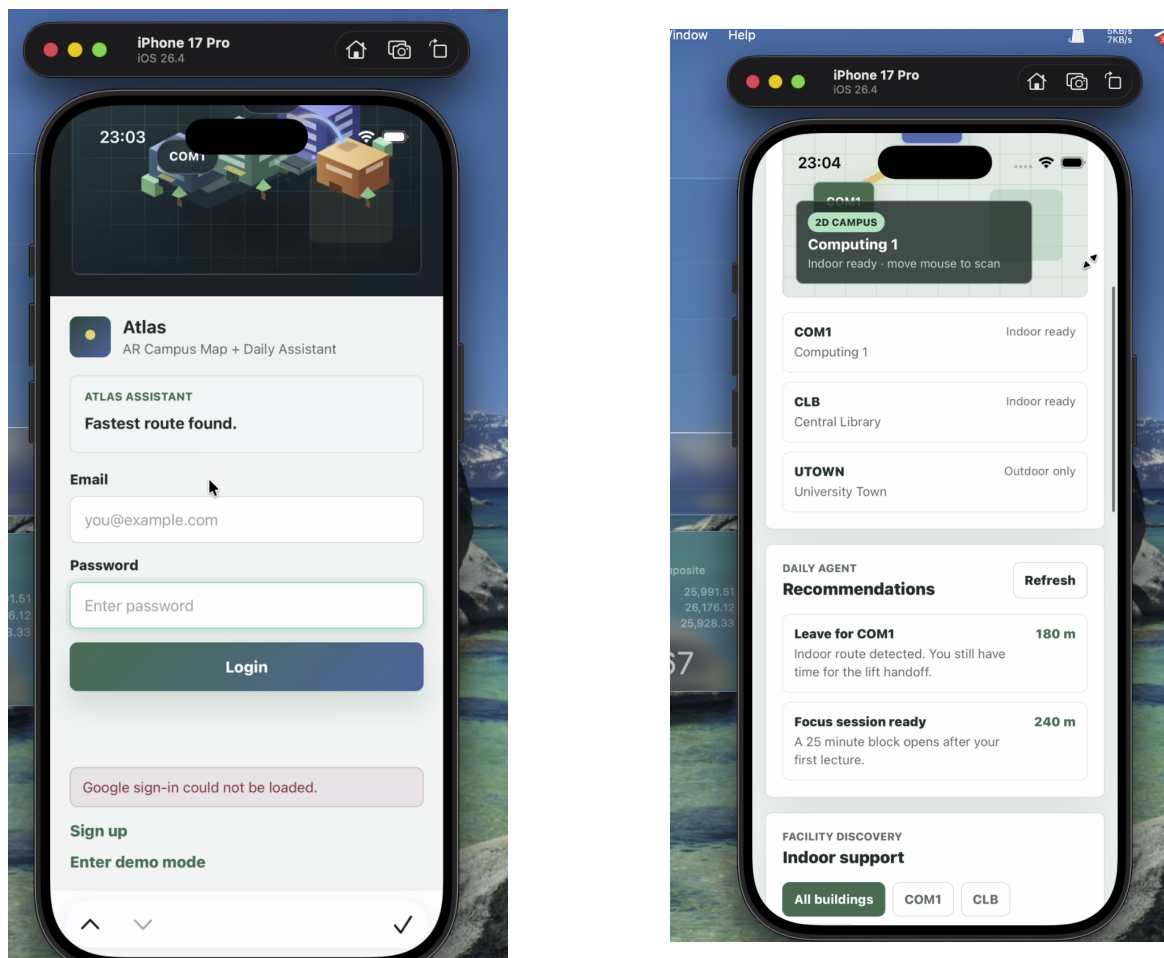
Bus API	Use Case
GET /api/bus/stops	List campus shuttle stops.
GET /api/bus/routes	List available NUS shuttle routes.
GET /api/bus/arrivals?stop=COM2	Show estimated arrivals for a selected stop.
GET /api/bus/active?route=D1	Show active vehicles for a route.
GET /api/bus/context?lat=...	Provide nearby transit context for assistant recommendations.

Mobile Frontend Experience

Because campus navigation is often used while students are moving, mobile usability is a supporting but important feature. Issue #10 tracks responsive layout and manual viewport checks for authentication, dashboard, map, schedule, and recommendations.

Responsive Layout

The frontend is designed to remain readable on common mobile widths. Cards, buttons, filters, and map panels are arranged to avoid horizontal scrolling and text overlap.



Mobile App Screens

The main engineering principle here is progressive enhancement: the application should remain usable on desktop while also adapting to mobile. The trade-off is extra CSS and layout testing, but this matters for a campus assistant because students are more likely to use it on phones than on large desktop screens.

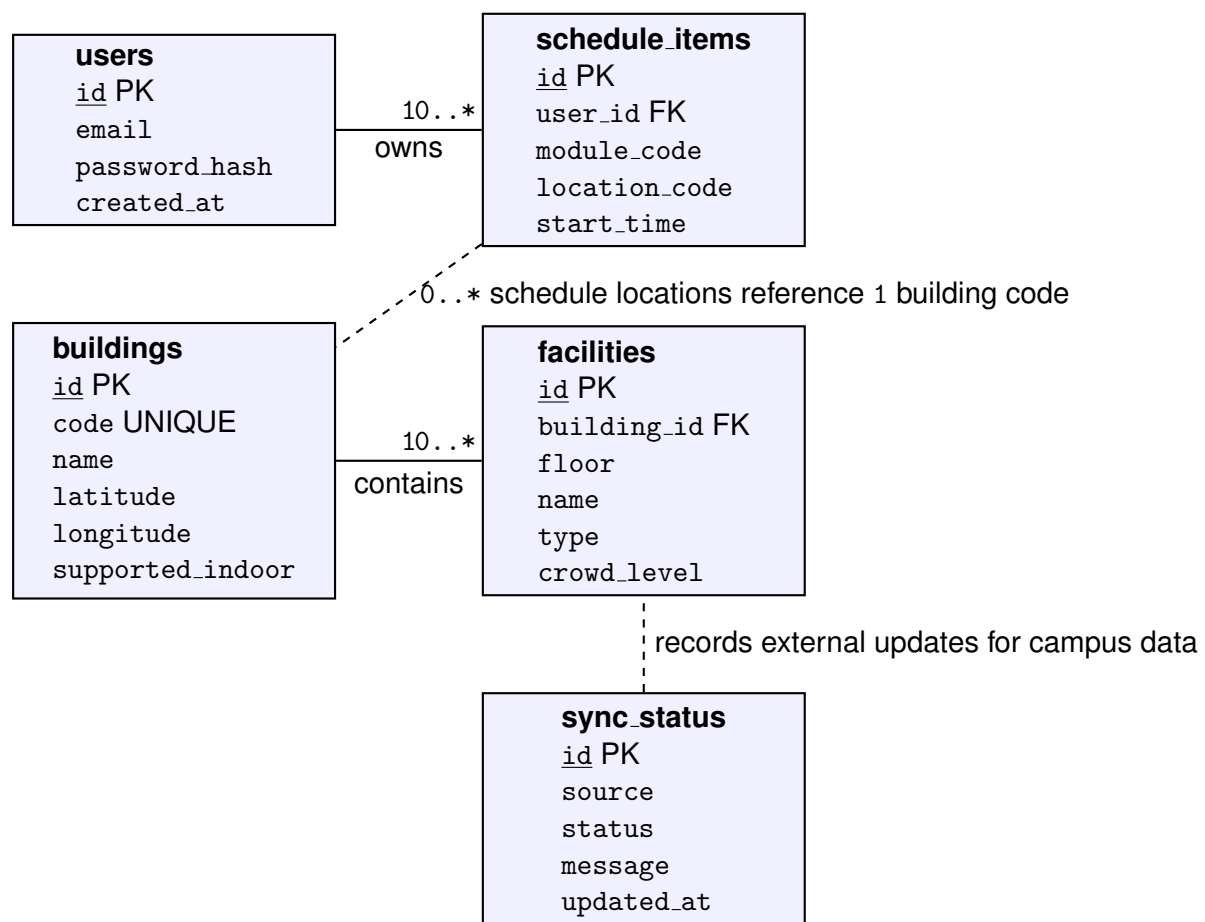
Standardised Software Diagrams

In addition to the high-level architecture diagram and the authentication sequence diagram, the following standardised diagrams describe persistent data relationships and the map/search user journey.

ER Diagram

The ER diagram models the core persistent data needed for Milestone 01 and later Artemis extensions. The notation below uses entity boxes with primary keys (PK), foreign keys (FK), and relationship cardinalities such as 1 and 0..*. Users own schedule items, buildings contain facilities, and sync status records document external data updates.

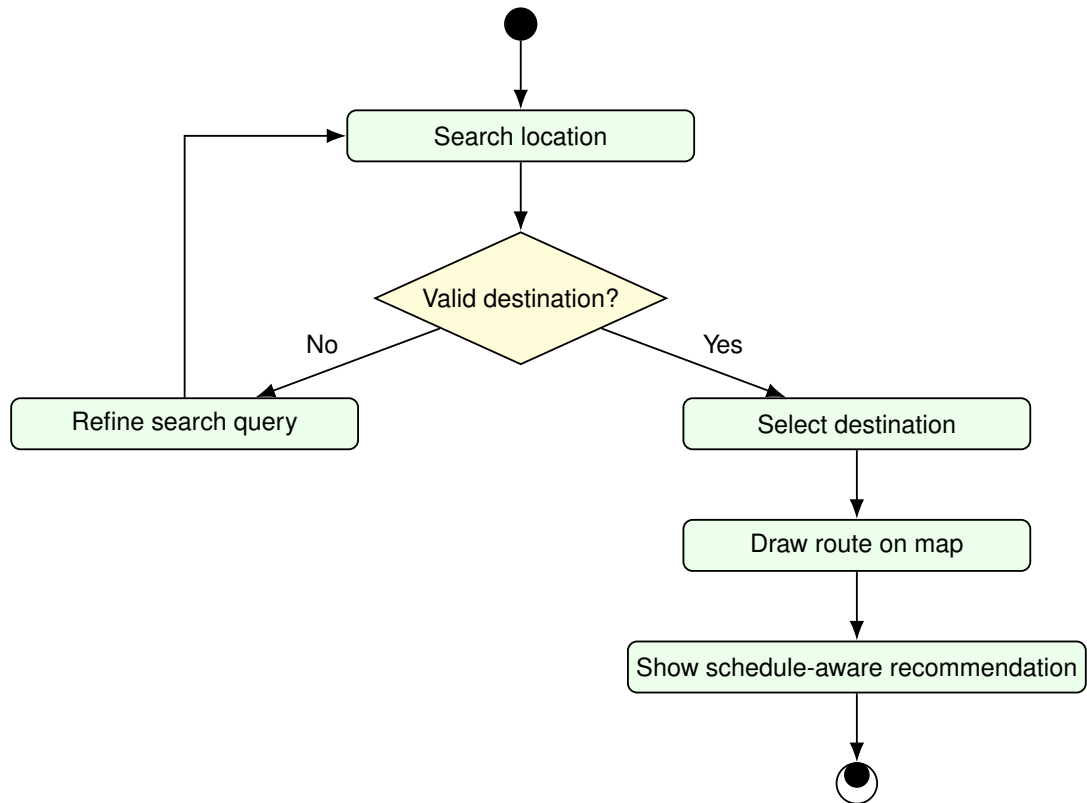
The database is designed to reduce unnecessary duplication. Building data is stored once in buildings, while facility-specific data is stored in facilities. User schedule records are stored separately from user credentials. This is close to Third Normal Form (3NF): each table stores facts about one main entity, and non-key attributes depend on the table's key rather than on another non-key attribute. This makes the schema easier to extend when later milestones add more facilities, indoor maps, timetable sync, or user preferences.



ER diagram for users, buildings, facilities, schedule items, and sync status

Activity Diagram

The activity diagram describes the user journey for map search and route-oriented recommendations. The flow starts from a location search, moves through destination selection and route rendering, and ends with contextual recommendations.



Activity diagram for search location, destination selection, route drawing, and recommendation display

Automated Testing and QA

Testing for Milestone 01 is split into automated checks and manual usability checks. Automated checks are used for repeatable engineering confidence, while manual checks are used where the current prototype depends on browser behavior, map rendering, OAuth, and mobile layout.

Automated Checks

Check	Command	What It Validates
Backend package tests	<code>cd backend && go test ./...</code>	Compiles backend packages and runs Go test files when present. This protects API handlers, store logic, auth logic, scheduler logic, and integration helpers from compile-time regressions.
Frontend production build	<code>cd frontend && npm run build</code>	Verifies that React, Vite, routing, TypeScript/JSX modules, and imported assets can produce a production bundle.
iOS shell sync build	<code>cd frontend && npm run ios:sync</code>	Runs the frontend build and synchronizes it with Capacitor iOS configuration for mobile demo validation.
Docker production build	<code>docker compose up -d --build</code>	Validates that the frontend and backend can be built into the production container and served through the configured deployment stack.

At this milestone, the strongest automated coverage is build-level and package-level validation. The next planned improvement is to add explicit unit tests for backend store/API behavior and frontend component tests for authentication, dashboard loading, schedule actions, and map UI state.

Manual Test Matrix

Flow Tested	Method	Expected Result
Demo login	Use seeded demo account test1@gmail.com / cp2106.	User reaches the protected dashboard and receives an application JWT.
Registration	Register a new account through the auth screen.	New credentials are accepted and can be used to access protected pages.
Google Sign-In	Sign in with a configured Google OAuth client.	Backend verifies the ID token and returns the same app JWT format.
Dashboard loading	Open protected dashboard after login.	Health, buildings, schedule, recommendations, facilities, and sync status appear.
Facility filtering	Change building and type filters.	Facility list updates without breaking dashboard layout.
Schedule actions	Add and delete schedule items.	Schedule list updates and recommendation context remains consistent.
Campus map	Open map page, search location, and draw route.	Map loads, marker/route interactions work, and the UI remains usable.
Mobile viewport	Test auth, dashboard, map, and mobile screenshots at narrow widths.	Text, cards, buttons, and map panels remain readable without horizontal scrolling.
Production app	Visit the Render deployment URL.	The deployed app loads over HTTPS and supports the same demonstration flow.

CI/CD Pipeline

The CI/CD pipeline is designed around GitHub as the source of truth and Render/Docker as the deployment path. Its purpose is to prevent unverified changes from reaching production and to make deployment reproducible.

Continuous Integration Gates

For pull requests and pushes to `main`, the intended GitHub Actions workflow runs stages that each protect against a specific project risk:

Pipeline Stage	Command / Action	Why This Stage Is Included
Checkout	Fetch repository source	Ensures CI tests the same version of frontend, backend, deployment, and documentation files that will be reviewed or merged.
Frontend dependency install	<code>npm ci</code>	Uses the lockfile to install repeatable dependencies, reducing “works on my machine” differences between teammates and CI.
Frontend build	<code>npm run build</code>	Catches React/Vite import errors, broken assets, and production bundling failures before deployment.
Backend tests	<code>go test ./...</code>	Checks Go package compilation and backend test coverage for API, auth, storage, scheduler, and integration helpers.
Docker build	<code>docker build .</code>	Confirms the app can be packaged in the same production-style environment used for deployment.
Deployment validation	Manual Render walkthrough	Confirms the deployed HTTPS app supports the actual demo flow, including auth, dashboard, map, schedule, and bus contexts.

The corresponding workflow structure is:

```
name: Atlas CI
on: [push, pull_request]
jobs:
  frontend-build: npm ci + npm run build
  backend-test: go test ./...
  docker-build: docker build .
```

CI workflow gates for pull requests and main branch updates

Continuous Deployment Path

Atlas currently uses a production deployment path built from the repository's `Dockerfile`, `docker-compose.yml`, Caddy configuration, and Render-hosted application URL. The deployment flow is:

1. Merge reviewed changes into `main`.
2. Build the frontend with Vite.
3. Build the Go backend binary.
4. Package the production app through the multi-stage Dockerfile.
5. Serve the app over HTTPS through the configured deployment host.
6. Validate the deployed app using the manual test matrix and video walkthrough flow.

Environment variables such as `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `DB_PATH`, `ALLOWED_ORIGIN`, and bus API credentials are kept outside source code. This keeps deployment configurable while avoiding accidental exposure of private credentials.

0.1 Unit Testing

Unit testing verifies that individual functions and components behave correctly in isolation. It is important because it helps detect bugs early, improves software reliability, and ensures that future changes do not break existing functionality. In this project, unit tests were designed using testing heuristics such as boundary value analysis and error guessing to cover normal cases, edge cases, and invalid inputs.

For this project, testing was carried out by verifying each component and function separately using unit tests. The tests checked normal behavior, edge cases, and invalid inputs to ensure the application responded correctly and did not crash. For example, `getSuggestions()` was tested to confirm that it returns location suggestions for valid queries and handles API errors safely, while `SelectList()` was tested to ensure that items are rendered correctly and the selected item is passed back through the callback.

Table 2: Unit Testing Summary for `getSuggestions()`

Test Case	Input	Expected Outcome
Valid Query	"NUS Computing"	Function returns a non-empty list of location suggestions and coordinates.
Empty String	""	Function returns an empty array or no suggestions without crashing.
Invalid Location Name	"asdfghjkl"	Function handles no-match results gracefully and returns an empty array.
Special Characters	"!!!@@@"	Function does not crash and handles malformed input safely.
Large Input String	Very long search text	Function remains stable and returns either valid results or an empty array.
Custom Result Size	<code>opts = 10</code>	Function requests and returns up to the specified number of suggestions.
API Failure	Mocked API/network error	Function catches the exception, logs the error, and returns an empty array.
Missing Features Field	Mocked response without features	Function returns an empty array via the fallback <code>features []</code> .

0.2 Software Design Principles

The application was developed using established software design principles to improve maintainability, readability, reusability, and scalability. Table 4 summarizes how key features of the system apply these principles.

Table 3: Unit Testing Summary for `SelectList()`

Test Case	Input	Expected Outcome
Valid Item List	<code>["apple", "orange", "banana"]</code>	All items are rendered as separate menu entries.
Single Item List	<code>["apple"]</code>	Exactly one menu entry is rendered correctly.
Empty Item List	<code>[]</code>	No menu entries are displayed and the component remains stable.
Item Selection	User clicks an item	The corresponding item is passed to the parent <code>onClick</code> callback.
Large Item List	50+ items	All items are rendered correctly without missing entries.
Undefined Item List	<code>items = undefined</code>	Testing reveals potential runtime errors due to the use of <code>items.map()</code> ; identifies need for defensive handling.
Missing Callback	<code>onClick = undefined</code>	Testing reveals potential runtime errors when an item is clicked; identifies need for optional callback handling.

Milestone 01 Progress

Milestone 01 focused on ideation, planning, initial prototyping, and establishing a deployed technical foundation. The current implementation provides a working React frontend, Go backend, SQLite data layer, authentication flow, dashboard, map page, campus data APIs, and Render deployment.

The implementation also validates the proposed modular system design. Frontend pages, backend handlers, authentication logic, campus data storage, and map features are separated into maintainable modules, making it easier to extend the project in later milestones.

Planned Next Steps

- Expand the navigation layer toward richer indoor guidance and AR-style interaction.
- Improve campus data coverage for buildings, rooms, facilities, and navigation points.
- Continue the Sprint 3 work on transit, multimodal routing, and route visualization as extension features.
- Gather user feedback from NUS students and refine the assistant workflow.

Table 4: Application of Software Design Principles

Feature	Design Principle	Description
<code>getSuggestions()</code>	Single Responsibility, Abstraction	Handles only the retrieval of location suggestions from the geocoding API, while hiding API implementation details from other components.
<code>SelectList</code>	Modularity, Reusability	Provides a reusable component for displaying selectable suggestions independently of the data source.
<code>RoutingForm</code>	Separation of Concerns, Modularity	Manages user inputs and routing interactions while delegating suggestion display and map functionality to dedicated components.
<code>MapEngine</code>	Encapsulation, Single Responsibility	Centralises all map-related operations, such as marker placement, route drawing, and map navigation, within a single class.
<code>drawRoute()</code>	Abstraction	Provides a simple interface for route rendering without exposing underlying MapLibre source and layer management details.
<code>track()</code> and <code>setLocation()</code>	Encapsulation	Encapsulate location marker creation and updates, hiding implementation details from the rest of the application.